



ANALYSIS REPORT

Transportation Management System - Group: 11

Course: Database Lab - 161353
Instructor: Ph.D. Nguyễn Thị Oanh
Students Hà Tuấn Bảo - 20235897
Tạ Hải Luân - 20235969
Trịnh Lương Việt – 20236012

TABLE OF CONTENTS

CHAPTER 1. BUSINESS DESCRIPTION.....	1
1.1 Business Context.....	1
1.2 Users	1
1.3 Business Flow	1
1.3.1 User Interaction Processes	1
1.3.2 Support and Administrative Processes.....	2
1.4 Operational Rules.....	3
1.4.1 Driver Rules	3
1.4.2 Passenger Rules.....	3
1.4.3 System Rules.....	3
CHAPTER 2. APPLICATION DESCRIPTION	4
2.1 Driver	4
2.1.1 Account Management	4
2.1.2 Availability & Status Control	4
2.1.3 Ride Operations.....	4
2.1.4 Earnings & Payments.....	4
2.1.5 Notifications & Support.....	4
2.2 Customer	4
2.2.1 Account Management	4
2.2.2 Ride Booking	4
2.2.3 Trip Experience.....	5
2.2.4 Payments & Transactions	5
2.2.5 Notifications & Support.....	5
2.3 Enterprise	5
2.3.1 Driver Management	5
2.3.2 Vehicle Management	5
2.3.3 Invoice Management.....	6
CHAPTER 3. ENTITY RELATIONSHIP DIAGRAM.....	7
3.1 Our system - Transport Management System	7
3.1.1 Entity Relationship Diagram.....	7
3.1.2 Relational Schema.....	7
3.2 Group 12's System – Hospital Management System	9

3.2.1	Entity Relationship Diagram.....	10
3.2.1	Relational Schema.....	10
CHAPTER 4.	DATABASE INITIALIZATION	11
4.1	Normalization	11
4.2	Table Creation and Data Modeling.....	11
4.3	Data Generation	11
CHAPTER 5.	SQL QUERY DESIGN AND IMPLEMENTATION	13
5.1	Database Administration & System-Level Queries.....	13
5.1.1	Secure Support Team View for Customer Complaints	13
5.2	Customer Queries.....	14
5.2.1	Customer Spending Between Two Dates (Function-based)	14
5.2.2	Customers Without Any Completed Trip	14
5.2.3	Customer Spending Summary	15
5.2.4	Top 20 Customers with Highest Failed Amount	16
5.2.5	Trips Completed but Missing Payment.....	17
5.2.6	Customers Without Feedback (Special Case).....	17
5.2.7	Monthly Spending Statistics	18
5.2.8	Customers Active in Last N Days.....	19
5.2.9	Customers Using Credit Card Frequently.....	19
5.2.10	Ride Request Behavior by Hour.....	20
5.3	Driver Queries.....	22
5.3.1	Driver Location Search	22
5.3.2	Driver Performance Statistics	23
5.3.3	Driver Rest Rule Checking	24
5.3.4	Speed Fraud Detection	26
5.3.5	Periodic Vehicle Maintenance Check.....	27
5.3.6	Average Fare Range Analysis.....	28
5.3.7	High-Performance Driver Identification.....	28
5.3.8	Find Replacement Drivers in Case of Vehicle Failure	28
5.3.9	Service Type Vehicle Filtering	28
5.3.10	Destination Autocomplete Search.....	29
5.3.11	Driver Performance Dashboard using Materialized View	29
5.4	Enterprise Queries.....	31
5.4.1.....		31

5.4.2.....	31
5.4.3.....	33
5.4.4.....	36
5.4.5.....	38
5.4.6.....	38
5.4.7.....	41
5.4.8.....	43
5.4.9.....	46
5.4.10.....	50

CHAPTER 1. BUSINESS DESCRIPTION

1.1 Business Context

The Transportation Management System is built to help companies manage their ride services more effectively using technology. It helps automate processes, handle bookings, manage trips, collect data, and check performance.

Goal: Our group will provide a solution to manage data more effectively and organize the database so operators can work efficiently and access useful information for the company

1.2 Users

- **Customers:** use the transportation service. They can create an account, book trips, make payments, and manage personal information. They can also check trip history, rate drivers, give feedback to improve service, or make complaints to solve problems.
- **Drivers:** provide the transportation service. They can find and accept suitable trips, confirm and follow the assigned route. They can also track income, manage work status, and update personal, vehicle, and bank information for payment.
- **Enterprise:** manages and controls the whole system. It monitors driver, vehicle, and customer data, handles bills and salaries, and tracks performance. It is also responsible for keeping operations transparent, maintaining stable revenue, and improving system efficiency.

1.3 Business Flow

1.3.1 User Interaction Processes

Case 1: Ride Booking Process The customer opens the app, enters the pickup and destination points, and submits a ride request. The system searches for nearby available drivers and assigns the most suitable one based on location and performance factors. The driver can accept the request, after which the customer receives real-time tracking and ETA updates. If the customer cancels, the system records the event and applies any relevant penalties according to company policy.

Case 2: Account Registration and Verification The user submits a registration form, and the system creates an account and sends an OTP for verification. Once the user verifies the OTP, the account becomes active. Drivers must upload required identification and vehicle documents, which are reviewed by an administrator. Only users with fully verified profiles can access booking or ride-acceptance features.

Case 3: Driver Ride Execution The driver receives a ride request and reviews the trip details. After accepting, the system updates the trip status and provides real-time tracking as the driver heads to the pickup point. The driver starts the trip at

pickup, and the system continuously logs movement data until the driver completes the ride at the destination. The system then calculates the final fare and sends a trip summary to both driver and passenger.

Case 4: Payment & Feedback Process Once the trip is completed, the system processes the passenger's selected payment method and issues a receipt. After payment confirmation, the passenger can submit a rating and feedback. This feedback is stored and used by the company for driver evaluation and service improvement.

Case 5: Driver Resignation Process The driver requests account deletion and confirms identity through password re-entry. The system deactivates the driver's account and anonymizes related historical records for data integrity. A deletion event is logged, and the driver receives a final confirmation before the session is closed.

Case 6: Handling Vehicle Breakdown During a Trip The driver reports a vehicle breakdown, and the system logs the incident details. A nearby available driver is automatically assigned to continue the trip, and both passenger and original driver are notified. The original driver's status is set to "Maintenance" until the issue is resolved and recorded in the maintenance system.

1.3.2 Support and Administrative Processes

Case 7: Handling Customer Feedback and Complaints The customer submits a complaint in the app, and the system logs the details for review. The support module analyzes related trip, payment, and feedback data to identify the issue. The support team updates the complaint status until it is resolved, and the customer receives a notification. All records are retained for analytics and service-quality improvement.

Case 8: Enterprise Monitoring and System Management

Administrators access a dashboard with real-time data on drivers, trips, and payments. The system calculates KPIs based on operational metrics such as:

- Driver performance – including the number of trips completed per day/week, average rating from customer feedback, cancellation rate, and on-time performance.
- Trip efficiency – average trip duration, average fare per trip, route utilization, and the percentage of trips completed versus requested.
- Financial metrics – total revenue, payment completion rate, and outstanding or delayed payments.
- Operational risks – unusual patterns such as sudden drops in driver activity, frequent trip cancellations, or abnormal fare discrepancies are automatically flagged

The system generates visual KPI reports (charts, tables, trends) and alerts administrators when thresholds are exceeded. Administrators' decisions, such as approving, investigating, or taking corrective actions, are recorded in the system to maintain transparency and ensure smooth operations.

1.4 Operational Rules

1.4.1 Driver Rules

- Drivers can only accept rides when marked as available.
- Requests are rejected if the driver is over 5 km away or has unstable GPS.
- After completing 10 trips within 3 consecutive hours, the system temporarily disables trip acceptance.
- Drivers cannot accept rides if the vehicle is under maintenance or the profile is unverified.
- Trips may only be canceled for valid reasons (technical issues, emergencies).
- More than 3 cancellations per day result in account warnings or suspension.

1.4.2 Passenger Rules

- Passengers may not cancel more than 3 trips within 24 hours; exceeding this limit restricts booking.
- New trips cannot be booked while another is active.
- Requests with identical pickup and drop-off points are rejected.
- Payment must be validated before the trip is marked as completed.
- Accounts with disputed or delayed payments will have that payment method temporarily disabled.

1.4.3 System Rules

- All actions (booking, cancellation, payment, feedback) are fully logged with timestamps and user IDs.
- The system prioritizes drivers with high completion rates and ratings.
- Abnormal data (e.g., zero distance or unrealistic duration) triggers automatic alerts to administrators.
- Trip states (booked, accepted, in-progress, completed) must follow sequential order.
- Payment data is saved immediately after transaction success and cannot be manually edited.
- Data integrity across all tables (passenger, driver, trip, payment) must be maintained.

CHAPTER 2. APPLICATION DESCRIPTION

Our system will provide function for three main entities: Enterprise, Driver, and Customer.

2.1 Driver

2.1.1 Account Management

- Sign Up: Create account, upload documents for verification.
- Login / Logout: Secure login/logout with password or OTP.
- Profile Management: Update personal, vehicle, and bank information.

2.1.2 Availability & Status Control

- Go Online/Offline: Toggle to receive or stop receiving requests.
- Location Tracking: Real-time GPS updates when online.

2.1.3 Ride Operations

- Receive Request: Get ride offer with pickup and destination details.
- Accept/Reject: Respond within time limit; timeout if ignored.
- Trip Execution: Follow route via map; update trip status.
- Complete Trip: End ride, confirm completion, and rate passenger.

2.1.4 Earnings & Payments

- View Earnings: Check daily, weekly, or monthly income summary.
- Withdraw Funds: Transfer balance to linked bank or wallet.

2.1.5 Notifications & Support

- Receive Alerts: Get ride, payment, and policy notifications.
- Contact Support: Report issues or ask for help via chat or call.

2.2 Customer

2.2.1 Account Management

- Sign Up: Create an account using phone number or email; verify identity via OTP or ID upload if required.
- Login / Logout: Secure authentication using password or OTP.
- Profile Management: Update personal details such as name, phone number, preferred payment method, and saved addresses (home, work, etc.).

2.2.2 Ride Booking

- Set Pickup & Destination: Choose pickup and drop-off locations using map or saved addresses.
- Ride Options: Select service type (e.g., Bike, Car, Premium) and view estimated fare and arrival time.
- Confirm Booking: Confirm ride request and wait for driver acceptance.
- Track Driver: View driver's real-time location and ETA after booking

confirmation.

- **Cancel Booking:** Option to cancel before or during driver assignment, subject to cancellation policy.

2.2.3 Trip Experience

- **In-Trip Updates:** View live route tracking, driver information, and estimated arrival time.
- **Communication:** Call or message the driver directly via in-app system.
- **Safety Features:** Use emergency contact button or share trip with friends/family.
- **Complete Trip:** Confirm trip completion and rate the driver.

2.2.4 Payments & Transactions

- **Fare Estimation:** Automatically calculated based on distance, traffic, and service type.
- **Payment Methods:** Pay via cash, credit/debit card, e-wallet, or in-app balance.
- **Receipts:** View and download digital receipts after each trip.
- **Trip History:** Access detailed history of all rides, including cost, route, and driver information.

2.2.5 Notifications & Support

- **Receive Alerts:** Get notifications for ride status, driver arrival, promotions, and payment confirmations.
- **Customer Support:** Report issues, lost items, or request refunds through in-app chat or call.
- **Feedback System:** Rate drivers and provide comments to improve service quality

2.3 Enterprise

2.3.1 Driver Management

- **Add/Update Profiles:** Register new drivers and modify existing data.
- **Verify Licenses:** Check driver's license and document validity.
- **Track Status:** Monitor driver availability and trip activity in real time.
- **Evaluate Performance:** Review ratings, complaints, and service quality

2.3.2 Vehicle Management

- **Register Vehicles:** Add and approve new vehicles in the system.
- **Update Info:** Edit vehicle details such as model, plate number, and ownership.
- **Insurance Monitoring:** Track insurance expiry dates and renewal status.
- **Maintenance Tracking:** Record and remind scheduled maintenance.

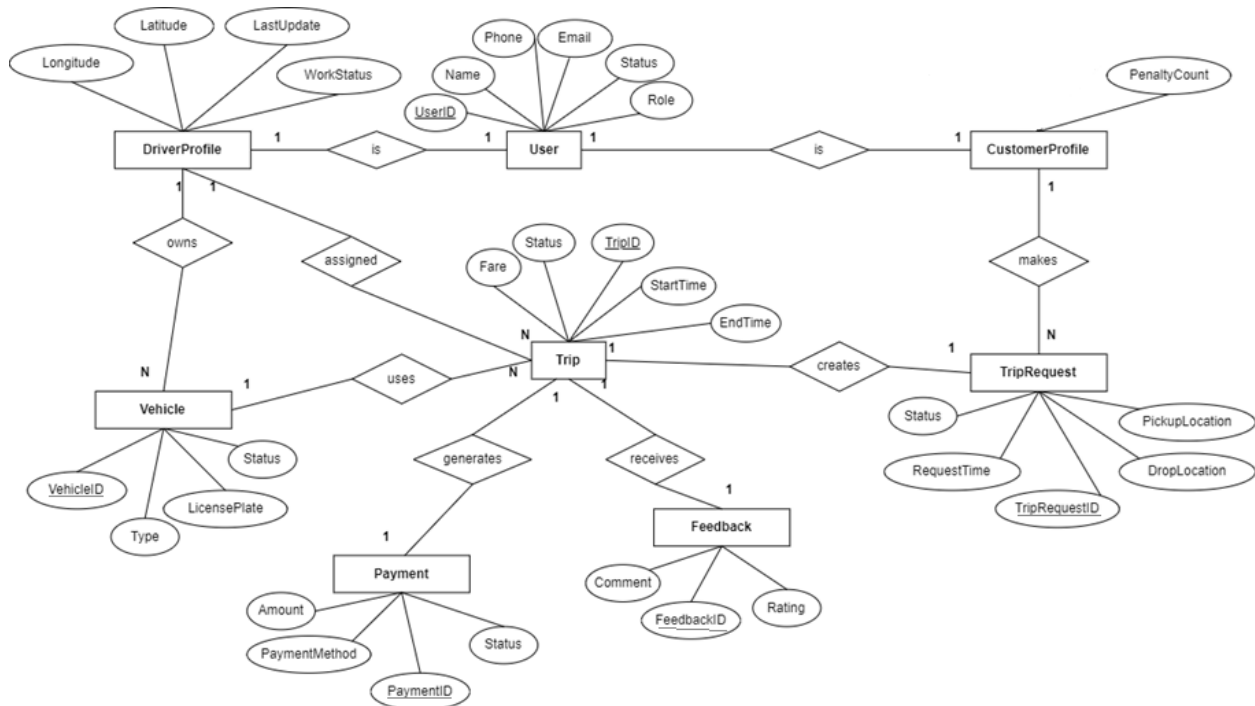
2.3.3 Invoice Management

- Record Transactions: Store details of all payments and ride revenues.
- Salary Payouts: Calculate and distribute driver earnings automatically.
- Revenue Reports: Generate financial summaries and analytics.
- Accuracy Checks: Validate payment amounts and detect inconsistencies

CHAPTER 3. ENTITY RELATIONSHIP DIAGRAM

3.1 Our system - Transport Management System

3.1.1 Entity Relationship Diagram



3.1.2 Relational Schema

- Table 'appuser':

Name	Type	Constraints
user_id	BIGINT	PK
name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	NOT NULL, UNIQUE
email	VARCHAR(100)	NOT NULL, UNIQUE
gender	INT	CHECK (0, 1)
role	VARCHAR(10)	NOT NULL, CHECK (customer, driver)
join_date	DATE	NOT NULL
status	VARCHAR(10)	NOT NULL, CHECK (active, inactive, suspended)

- Table 'customer':

Name	Type	Constraints
user_id	BIGINT	PK, FK → appuser(user_id)
penalty_count	INT	DEFAULT 0, CHECK ≥ 0

- Table 'driver':

Name	Type	Constraints
user_id	BIGINT	PK, FK → appuser(user_id)
work_status	VARCHAR(15)	NOT NULL, CHECK (available, unavailable)
longitude	DECIMAL(9,6)	NOT NULL
latitude	DECIMAL(8,6)	NOT NULL
last_update	TIMESTAMP	NOT NULL

- Table 'vehicle':

Name	Type	Constraints
vehicle_id	BIGINT	PK
license_plate	VARCHAR(20)	NOT NULL, UNIQUE
type	VARCHAR(10)	NOT NULL, CHECK (car, motorbike)
status	VARCHAR(15)	NOT NULL, CHECK (active, maintenance)
driver_id	BIGINT	FK → driver(user_id)

- Table 'trip_request':

Name	Type	Constraints
trip_request_id	BIGINT	PK
user_id	BIGINT	FK → customer(user_id)
pickup_location	TEXT	NOT NULL
drop_location	TEXT	NOT NULL
pickup_latitude	DECIMAL(8,6)	NOT NULL
pickup_longitude	DECIMAL(9,6)	NOT NULL
drop_latitude	DECIMAL(8,6)	NOT NULL
drop_longitude	DECIMAL(9,6)	NOT NULL
status	VARCHAR(10)	NOT NULL, CHECK (requested, rejected, accepted)
request_time	TIMESTAMP	NOT NULL

- Table 'trip':

Name	Type	Constraints
------	------	-------------

Name	Type	Constraints
trip_id	BIGINT	PK
trip_request_id	BIGINT	UNIQUE, FK → trip_request(trip_request_id)
driver_id	BIGINT	FK → driver(user_id)
vehicle_id	BIGINT	FK → vehicle(vehicle_id)
status	VARCHAR(15)	NOT NULL, CHECK (assigned, in_progress, completed, canceled)
start_time	TIMESTAMP	NULL
end_time	TIMESTAMP	NULL
fare	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0

- Table ‘payment’:

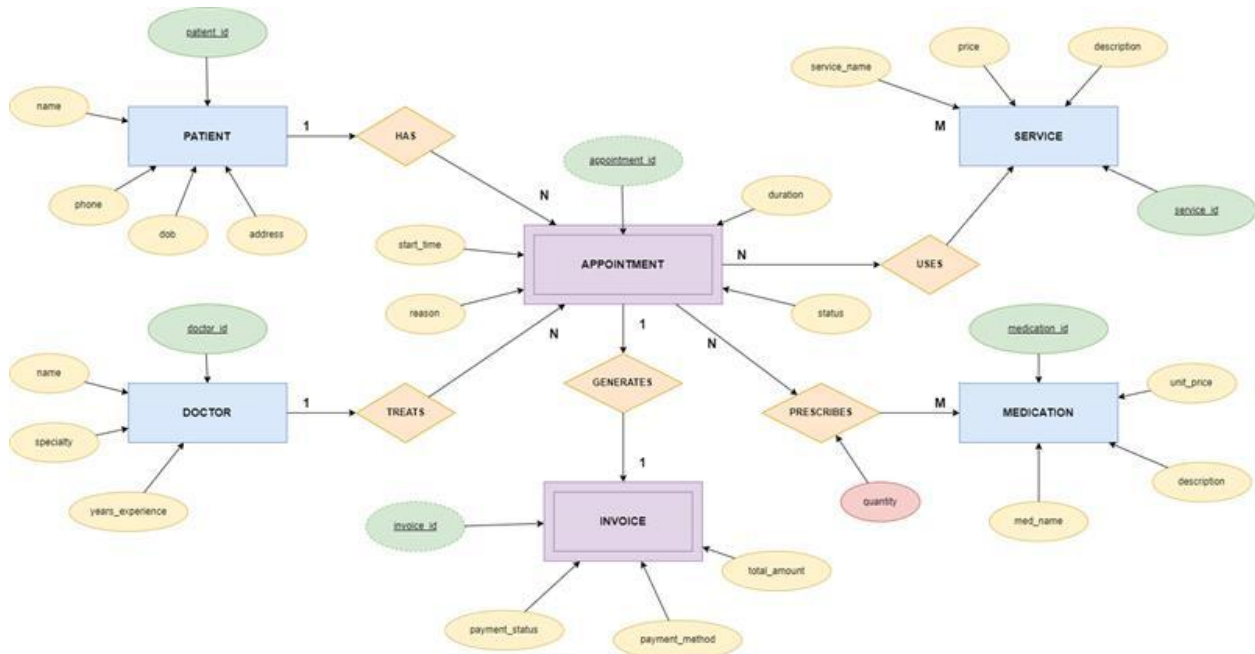
Name	Type	Constraints
payment_id	BIGINT	PK
trip_id	BIGINT	UNIQUE, FK → trip(trip_id)
amount	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0
payment_method	VARCHAR(10)	NOT NULL, CHECK (cash, wallet, credit)
status	VARCHAR(10)	NOT NULL, CHECK (completed, failed)

- Table ‘feedback’:

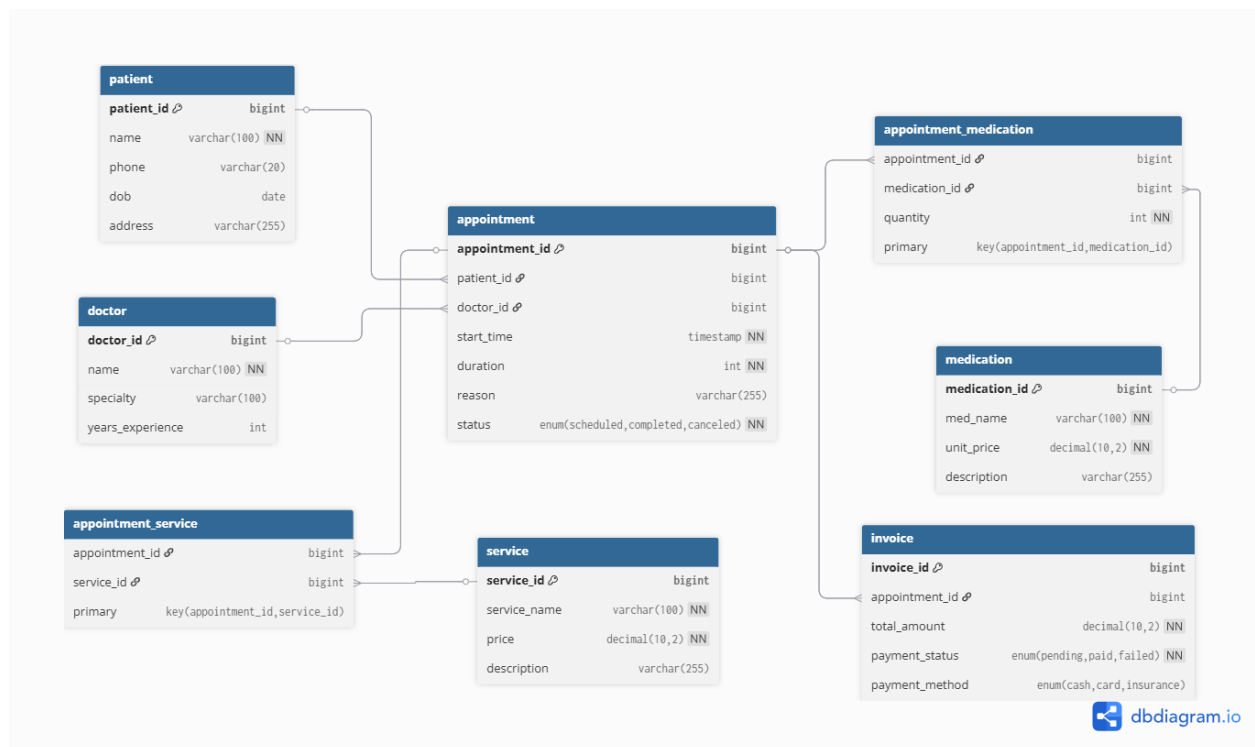
Name	Type	Constraints
feedback_id	BIGINT	PK
trip_id	BIGINT	UNIQUE, FK → trip(trip_id)
rating	SMALLINT	NOT NULL, CHECK (1–5)
comment	VARCHAR(512)	NULL

3.2 Group 12’s System – Hospital Management System

3.2.1 Entity Relationship Diagram



3.2.1 Relational Schema



CHAPTER 4. DATABASE INITIALIZATION

This chapter presents the process of initializing the TMS database schema. It begins with normalization analysis to ensure a well-structured relational design, followed by table creation with appropriate keys and constraints. Finally, initial data is generated and validated to populate the database while preserving data integrity, business rules, and consistency across all relations.

4.1 Normalization

- **appuser** (Key: *user_id*, FD: *user_id* → *name*, *phone*, *email*, *gender*, *role*, *join_date*, *status*) All non-key attributes depend directly on the primary key, with no partial or transitive dependency → 3NF
- **customer** (Key: *user_id*, FD: *user_id* → *penalty_count*) Single-attribute primary key, no transitive dependency → 3NF
- **driver** (Key: *user_id*, FD: *user_id* → *work_status*, *longitude*, *latitude*, *last_update*) All attributes are fully functionally dependent on the primary key → 3NF
- **vehicle** (Key: *vehicle_id*, FD: *vehicle_id* → *license_plate*, *type*, *status*, *driver_id*) No non-key attribute determines another non-key attribute → 3NF
- **trip_request** (Key: *trip_request_id*, FD: *trip_request_id* → *user_id*, *pickup_location*, *drop_location*, *pickup_latitude*, *pickup_longitude*, *drop_latitude*, *drop_longitude*, *status*, *request_time*) All attributes depend only on the request identifier → 3NF
- **trip** (Key: *trip_id*, FD: *trip_id* → *trip_request_id*, *driver_id*, *vehicle_id*, *status*, *start_time*, *end_time*, *fare*) No partial or transitive dependency among non-key attributes → 3NF
- **payment** (Key: *payment_id*, FD: *payment_id* → *trip_id*, *amount*, *payment_method*, *status*) Payment information depends solely on the primary key → 3NF
- **feedback** (Key: *feedback_id*, FD: *feedback_id* → *trip_id*, *rating*, *comment*) Rating and comment are directly dependent on the feedback identifier → 3NF

4.2 Table Creation and Data Modeling

Detail in: https://github.com/rabituckle/tms_db_initialization

4.3 Data Generation

Method: Use Python (*Faker* + *random*) and Export SQL INSERT per table

Process:

- Generate data by dependency order
- Apply business rules during generation
- Control ratios using *random.choices*

Validation and Integration:

We use *pytest* to validate the generated data, including foreign key existence, ratio distribution, and value consistency. The SQL files are parsed using regular expressions to avoid errors caused by commas in text fields. Finally, all SQL statements are merged into a single data_tms.sql file and wrapped in a transaction for safe and atomic import.

Data Constraints:

Table	Main Business Constraints	Ratio / Distribution
appuser	Unique user_id; role $\in$ {customer, driver}; join_date $\leq$ current date	role: customer 80%, driver 20% status: active 88%, inactive 10%, suspended 2%
customer	Generated only from appuser with role = customer	penalty_count: 0 (70%), 1 (20%), 2 (7%), ≥ 3 (3%)
driver	Generated only from active appuser with role = driver	work_status: available 80%, unavailable 20% last_update: available $\leq$ 4h, unavailable 1–2 days
vehicle	One vehicle per driver	type: motorbike 70%, car 30% status: mostly active
trip_request	user_id must be customer; request_time $\geq$ join_date; IDs increase over time; pickup–drop distance limited	status: requested / accepted / rejected (all present) 5% customers have no requests 1% customers have >120 requests
trip	Each trip linked to one trip_request; start_time $\leq$ end_time (if any); fare ≥ 0	status: assigned / in_progress / completed / canceled fare: car $>$ motorbike
payment	Created only for completed trips; amount = fare	method: cash 50%, wallet 40%, credit 10% status: completed 95%, failed 5%
feedback	Created only for completed or canceled trips	$\sim 80\%$ with rating $\sim 30\%$ with rating + comment canceled: mostly low ratings ($\sim 8:2$) completed: rating ratio 1:1:3:2:3

CHAPTER 5. SQL QUERY DESIGN AND IMPLEMENTATION

5.1 Database Administration & System-Level Queries

5.1.1 Secure Support Team View for Customer Complaints

Requirement: Allow the Support Team to access contact information only for customers with complaints (1–2 star ratings), without granting access to the full appuser table.

Business Logic: Create a restricted database view that joins customer, trip, and feedback data, returning only customers who have submitted low ratings and require support intervention.

Query:

```
CREATE VIEW view_support_customer_contact AS
SELECT
    u.user_id,
    u.name,
    u.phone,
    u.email,
    f.rating,
    f.comment
FROM appuser u
JOIN trip_request tr ON u.user_id = tr.user_id
JOIN trip t ON tr.trip_request_id = t.trip_request_id
JOIN feedback f ON t.trip_id = f.trip_id
WHERE u.role = 'customer'
    AND f.rating IN (1, 2);
```

Principle of Least Privilege: Support staff cannot access the appuser table directly; they can only view specific customer data through a restricted View.

Optimization: Using f.rating IN (1, 2) allows the database to utilize the index on the rating column more effectively than an OR operator.

High Performance: By joining tables via Foreign Keys, the query ensures Index Seek operations instead of performing a full table scan.

5.2 Customer Queries

5.2.1 Customer Spending Between Two Dates (Function-based)

Requirement

Allow a customer to query total spending between two dates (e.g. “How much did I spend last month?”).

Business Logic

- Only completed trips
- Only completed payments
- Filter by customer and date range
- Return 0 instead of NULL if no trips

User-defined Function

```
CREATE OR REPLACE FUNCTION fn_customer_total_spending(  
    p_user_id    BIGINT,  
    p_start_date DATE,  
    p_end_date   DATE  
)  
RETURNS NUMERIC AS $$  
DECLARE total NUMERIC;  
BEGIN  
    SELECT COALESCE(SUM(p.amount), 0)  
    INTO total  
    FROM trip_request tr  
    JOIN trip t ON t.trip_request_id = tr.trip_request_id  
    JOIN payment p ON p.trip_id = t.trip_id  
    WHERE tr.user_id = p_user_id  
        AND t.status = 'completed'  
        AND p.status = 'completed'  
        AND t.end_time >= p_start_date  
        AND t.end_time < p_end_date + INTERVAL '1 day';  
    RETURN total;  
END;  
$$ LANGUAGE plpgsql;
```

Optimization Techniques

- Encapsulation via FUNCTION
- Status filtering
- Date range filtering
- COALESCE avoids NULL

5.2.2 Customers Without Any Completed Trip

Requirement

Find customers who never successfully completed a trip.

Business Logic

- A customer is invalid if they appear in any completed trip
- Anti-join logic required

Naive Query

```
SELECT c.user_id
FROM customer c
WHERE c.user_id NOT IN (
    SELECT tr.user_id
    FROM trip t
    JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
    WHERE t.status = 'completed'
);
```

Optimized Query

```
SELECT c.user_id
FROM customer c
LEFT JOIN (
    SELECT DISTINCT tr.user_id
    FROM trip t
    JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
    WHERE t.status = 'completed'
) completed_users
ON c.user_id = completed_users.user_id
WHERE completed_users.user_id IS NULL;
```

Optimization Techniques

- Replace NOT IN with LEFT JOIN ... IS NULL
- Avoid NULL pitfalls
- PostgreSQL optimizes anti-joins better

5.2.3 Customer Spending Summary Requirement

Calculate total money spent by each customer.

Business Logic

- Only completed payments
- Aggregate per customer
- Ignore zero-spending customers

Naive Query

```
SELECT tr.user_id, SUM(p.amount)
FROM payment p
JOIN trip t ON p.trip_id = t.trip_id
```

```
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE p.status = 'completed'
GROUP BY tr.user_id;
```

Optimized Query

```
SELECT tr.user_id, SUM(p.amount) AS total_spent
FROM payment p
JOIN trip t ON p.trip_id = t.trip_id
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE p.status = 'completed'
GROUP BY tr.user_id
HAVING SUM(p.amount) > 0;
```

Optimization Techniques

- Partial index on completed payments
- HAVING clause filters irrelevant groups
- Impact limited due to small dataset

5.2.4 Top 20 Customers with Highest Failed Amount Requirement

Identify top 20 customers with the highest total failed payment amount.

Business Logic

- Focus on financial impact
- Only failed payments
- Aggregate by customer

Naive Query

```
SELECT au.user_id, au.name, au.email, au.phone,
       COUNT(*) AS failed_payment_count,
       SUM(p.amount) AS failed_amount
FROM payment p
JOIN trip t ON p.trip_id = t.trip_id
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
JOIN appuser au ON tr.user_id = au.user_id
WHERE p.status = 'failed'
GROUP BY au.user_id, au.name, au.email, au.phone
ORDER BY failed_amount DESC
LIMIT 20;
```

Optimized Query

```
SELECT au.user_id, au.name, au.email, au.phone,
       COUNT(p.payment_id) AS failed_payment_count,
       ROUND(SUM(p.amount), 2) AS failed_amount,
       MAX(t.end_time) AS last_failure_date
FROM payment p
```

```

JOIN trip t ON p.trip_id = t.trip_id
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
JOIN appuser au ON tr.user_id = au.user_id
WHERE p.status = 'failed'
GROUP BY au.user_id, au.name, au.email, au.phone
HAVING SUM(p.amount) > 0
ORDER BY failed_amount DESC
LIMIT 20;

```

Optimization Techniques

- Partial index on failed payments
- LIMIT reduces sort cost

5.2.5 Trips Completed but Missing Payment Requirement

Detect completed trips without payment.

Business Logic

- Completed trip must have exactly one payment
- Missing payment indicates system error

Naive Query

```

SELECT t.trip_id, t.end_time, t.fare
FROM trip t
WHERE t.status = 'completed'
      AND t.trip_id NOT IN (
          SELECT p.trip_id FROM payment p
      );

```

Optimized Query

```

SELECT t.trip_id, t.end_time, t.fare
FROM trip t
LEFT JOIN payment p ON p.trip_id = t.trip_id
WHERE t.status = 'completed'
      AND p.trip_id IS NULL;

```

Optimization Techniques

- Anti-join pattern
- Avoid NULL pitfalls

5.2.6 Customers Without Feedback (Special Case) Requirement

Identify customers who never left feedback.

Naive Query

```
SELECT DISTINCT tr.user_id
FROM trip t
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE t.trip_id NOT IN (
    SELECT trip_id FROM feedback
);
```

Optimized Query

```
SELECT DISTINCT tr.user_id
FROM trip t
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
LEFT JOIN feedback f ON t.trip_id = f.trip_id
WHERE f.trip_id IS NULL;
```

Execution Plan Insight

- NOT IN rewritten as hashed subplan
- Lower cost than Hash Anti Join in this dataset

Conclusion

Naive query is faster in practice due to PostgreSQL planner optimization.

5.2.7 Monthly Spending Statistics

Requirement

View monthly spending and trip counts in a year.

Business Logic

- Group by month
- Only completed trips
- Filter by year and customer

Naive Query

```
SELECT EXTRACT(MONTH FROM t.end_time) AS month,
       COUNT(*) AS total_trips,
       SUM(t.fare) AS total_spent
FROM trip t
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE tr.user_id = 1500
      AND t.status = 'completed'
      AND EXTRACT(YEAR FROM t.end_time) = 2025
GROUP BY EXTRACT(MONTH FROM t.end_time)
ORDER BY month;
```

Optimized Query

```

SELECT DATE_TRUNC('month', t.end_time) AS month,
       COUNT(*) AS total_trips,
       SUM(t.fare) AS total_spent,
       AVG(t.fare) AS avg_fare
FROM trip t
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE tr.user_id = 1500
      AND t.status = 'completed'
      AND t.end_time >= '2025-01-01'
      AND t.end_time < '2026-01-01'
GROUP BY DATE_TRUNC('month', t.end_time)
ORDER BY month;

```

Optimization Techniques

- DATE_TRUNC instead of EXTRACT
- Range scan on date

5.2.8 Customers Active in Last N Days

Requirement

Identify customers active recently.

Naive Query

```

SELECT DISTINCT tr.user_id
FROM trip t
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE t.end_time >= NOW() - INTERVAL '30 days';

```

Optimized Query

```

SELECT DISTINCT tr.user_id
FROM trip t
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE t.status = 'completed'
      AND t.end_time >= CURRENT_DATE - 30;

```

Optimization Techniques

- Status filter
- Partial index on completed trips

5.2.9 Customers Using Credit Card Frequently

Requirement

Detect customers who prefer credit card payments.

Naive Query

```

SELECT tr.user_id, COUNT(*)
FROM payment p

```

```

JOIN trip t ON p.trip_id = t.trip_id
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE p.payment_method = 'credit'
GROUP BY tr.user_id;

```

Optimized Query

```

SELECT tr.user_id, COUNT(*) AS credit_count
FROM payment p
JOIN trip t ON p.trip_id = t.trip_id
JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
WHERE p.payment_method = 'credit'
      AND p.status = 'completed'
GROUP BY tr.user_id
HAVING COUNT(*) >= 2;

```

Optimization Techniques

- Composite partial index
- HAVING filters noise

5.2.10 Ride Request Behavior by Hour Requirement

Analyze ride request behavior by hour and success rate.

Naive Query

```

SELECT EXTRACT(HOUR FROM request_time) AS hour_of_day,
       COUNT(*) AS total_requests,
       SUM(CASE WHEN status = 'accepted' THEN 1 ELSE 0 END) AS accepted,
       SUM(CASE WHEN status = 'rejected' THEN 1 ELSE 0 END) AS rejected
FROM trip_request
WHERE user_id = 1500
GROUP BY EXTRACT(HOUR FROM request_time)
ORDER BY hour_of_day;

```

Optimized Query

```

WITH hourly_stats AS (
    SELECT EXTRACT(HOUR FROM request_time) AS hour_of_day,
           status, COUNT(*) AS request_count
    FROM trip_request
    WHERE user_id = 1500
           AND request_time >= CURRENT_DATE - INTERVAL '90 days'
    GROUP BY EXTRACT(HOUR FROM request_time), status
)
SELECT hour_of_day,
       SUM(request_count) AS total_requests,
       SUM(CASE WHEN status = 'accepted' THEN request_count ELSE 0 END) AS
accepted,
       SUM(CASE WHEN status = 'rejected' THEN request_count ELSE 0 END) AS
rejected,
       ROUND(100.0 * SUM(CASE WHEN status = 'accepted' THEN request_count

```



```
ELSE 0 END)  
        / NULLIF(SUM(request_count), 0), 2) AS acceptance_rate  
FROM hourly_stats  
GROUP BY hour_of_day  
ORDER BY hour_of_day;
```

Optimization Techniques

- Two-phase aggregation using CTE
- Time window filtering
- Composite index on (user_id, request_time, status)

5.3 Driver Queries

5.3.1 Driver Location Search

Requirement: Find the top 5 nearest available drivers within a given radius.

Constraints: work_status = 'available', vehicle.status = 'active', optional vehicle type filter.

Data scale: ~8.900 active drivers, query executed frequently.

Naive Approach: Use Haversine formula:

$$d = 2r \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right)$$

SELECT

```
d.user_id AS driver_id,
u.name,
(6371 * asin(
  cos(radians(53.229225))
  * cos(radians(d.latitude))
  * sin(radians(longitude) -
radians(-2.179150))
  * sin(radians(longitude) -
radians(-2.179150))
  + sin(radians(53.229225)-
radians(d.latitude))
  * sin(radians(53.229225)-
radians(d.latitude))
)) AS distance_km
FROM driver d
JOIN vehicle v ON d.user_id = v.driver_id
JOIN appuser u ON u.user_id = d.user_id
WHERE d.work_status = 'available'
AND v.status = 'active'
AND d.latitude BETWEEN (latitude -
0.001) AND (latitude + 0.001)
AND d.longitude BETWEEN (longitude -
0.001) AND (longitude + 0.001)
ORDER BY distance_km
LIMIT 5;
```

```
tms=# explain analyze SELECT
tms=#   d.user_id AS driver_id,
tms=#   u.name,
tms=#   (6371 * acos(
tms=#     cos(radians(53.229225))
tms=#     * cos(radians(d.latitude))
tms=#     * cos(radians(longitude) - radians(-2.179150))
tms=#     + sin(radians(53.229225))
tms=#     * sin(radians(d.latitude))
tms=#   )) AS distance_km
tms=# FROM driver d
tms=# JOIN vehicle v ON d.user_id = v.driver_id
tms=# JOIN appuser u ON u.user_id = d.user_id
tms=# WHERE d.work_status = 'available'
tms=# AND v.status = 'active'
tms=# ORDER BY distance_km
tms=# LIMIT 10;

QUERY PLAN

Limit (cost=3140.18..3140.20 rows=10 width=30) (actual time=10.065..10.067 rows=10 loops=1)
  -> Sort (cost=3140.18..3154.43 rows=5700 width=30) (actual time=10.064..10.066 rows=10 loops=1)
    Sort Key: (((6371)::double precision * acos((((0.5986150895768312)::double precision * cos(radians(d.latitude))) * cos((radians(d.longitude) - '-0.038033342394834434'::double precision)))) + ('0.8010368122195898'::double precision * sin(radians(d.latitude))))))
    Sort Method: top-N heapsort  Memory: 26kB
    -> Hash Join (cost=691.74..3817.00 rows=5700 width=30) (actual time=2.610..9.570 rows=5671 loops=1)
      Hash Cond: (d.user_id = v.driver_id)
      -> Hash Join (cost=328.49..2384.75 rows=7134 width=46) (actual time=1.430..6.854 rows=7134 loops=1)
        Hash Cond: (u.user_id = d.user_id)
        -> Seq Scan on appuser u (cost=0.00..1925.00 rows=50000 width=22) (actual time=0.133..1.953 rows=50000 loops=1)
        -> Hash (cost=239.31..239.31 rows=7134 width=24) (actual time=1.270..1.271 rows=7134 loops=1)
          Buckets: 8192 Batches: 1 Memory Usage: 455kB
          -> Seq Scan on driver d (cost=0.00..239.31 rows=7134 width=24) (actual time=0.006..0.767 rows=7134 loops=1)
            Filter: ((work_status)::text = 'available'::text)
            Rows Removed by Filter: 1771
          -> Hash (cost=274.31..274.31 rows=7115 width=8) (actual time=1.150..1.151 rows=7115 loops=1)
            Buckets: 8192 Batches: 1 Memory Usage: 342kB
            -> Seq Scan on vehicle v (cost=0.00..274.31 rows=7115 width=8) (actual time=0.026..0.732 rows=7115 loops=1)
              Filter: ((status)::text = 'active'::text)
              Rows Removed by Filter: 1790
      Planning Time: 0.349 ms
      Execution Time: 10.179 ms
(21 rows)
```

→ Must perform sequential scan on 3 tables with approximately 50k + 7134 + 7115 rows ≈ 64000 rows, combined with hash join makes the computation scale increase. In reality, only a few drivers are concentrated in one area; adding a filter will help reduce the number of rows that need to be calculated.

```
AND d.latitude BETWEEN (53.229225 - 0.1) AND (53.229225 + 0.1)
AND d.longitude BETWEEN (-2.179150 - 0.1) AND (-2.179150 + 0.1)
```

```
Planning Time: 1.162 ms
Execution Time: 5.916 ms
(20 rows)
```

Optimize Approach: Spatial Indexing with Earthdistance

- **Spatial Indexing (GiST):** uses a GiST index with ll_to_earth to organize locations as 3D points. PostgreSQL quickly focus on nearby areas and find the closest results without reading the whole table.
- **Pre-filtering with Bounding Box (earth_box):** Before, the system calculated distances for more than 50,000 rows, earth_box with the @> operator lets the index quickly ignore drivers that are far away
- **KNN Distance Operator (<->):** With the <-> operator, the database reads results directly from the index already ordered by distance

```
CREATE EXTENSION IF NOT
EXISTS cube;

CREATE EXTENSION IF NOT
EXISTS earthdistance;

CREATE INDEX
idx_driver_location_gist ON driver
USING GIST (ll_to_earth(latitude,
longitude));

SELECT
    d.user_id AS driver_id, u.name,
    (ll_to_earth(d.latitude, d.longitude)
    <->
    ll_to_earth(53.229225, -2.179150))
    / 1000.0 AS distance_km
FROM driver d
JOIN vehicle v ON v.driver_id =
d.user_id
JOIN appuser u ON u.user_id =
d.user_id
WHERE
    d.work_status = 'available'
    AND v.status = 'active'
    AND
    earth_box(ll_to_earth(53.229225, -
2.179150), 50000)
    @> ll_to_earth(d.latitude,
d.longitude)
ORDER BY distance_km
LIMIT 5;
```

```
tms=# explain analyze SELECT
tms=#   d.user_id AS driver_id, u.name,
tms=#   (ll_to_earth(d.latitude::float8, d.longitude::float8)
tms=#   <->
tms=#   ll_to_earth(53.229225::float8, -2.179150::float8)) / 1000.0 AS distance_km
tms=# FROM driver d
tms=# JOIN vehicle v ON v.driver_id = d.user_id
tms=# JOIN appuser u ON u.user_id = d.user_id
tms=# WHERE
tms=#   d.work_status = 'available'
tms=#   AND v.status = 'active'
tms=#   AND earth_box(ll_to_earth(53.229225::float8, -2.179150::float8), 50000) -- Bán kính 50km
tms=#   @> ll_to_earth(d.latitude::float8, d.longitude::float8)
tms=# ORDER BY distance_km
tms=# LIMIT 5;

QUERY PLAN
-----
Limit  (cost=337.26..337.27 rows=5 width=30) (actual time=2.685..2.687 rows=5 loops=1)
-> Sort  (cost=337.26..337.28 rows=6 width=30) (actual time=2.684..2.685 rows=5 loops=1)
    Sort Key: (((ll_to_earth(d.latitude, d.longitude))::cube <-> '(3815306.4570526285, -145178.865683
89813, 5109147.3625209965)::cube) / '1000)::double precision) Sort Method: top-N heapsort  Memory
: 25kB
-> Nested Loop  (cost=35.82..337.18 rows=6 width=30) (actual time=0.418..2.646 rows=210 loops=1)
    -> Hash Join  (cost=35.53..328.53 rows=6 width=32) (actual time=0.305..1.500 rows=210 loops
=1)
        Hash Cond: (v.driver_id = d.user_id)
        -> Seq Scan on vehicle v  (cost=0.00..274.31 rows=7115 width=8) (actual time=0.050..0
.838 rows=7115 loops=1)
            Filter: ((status)::text = 'active'::text)
            Rows Removed by Filter: 1790
        -> Hash  (cost=35.44..35.44 rows=7 width=24) (actual time=0.242..0.243 rows=273 loops
=1)
            Buckets: 1024  Batches: 1  Memory Usage: 23kB
            -> Bitmap Heap Scan on driver d  (cost=4.22..35.44 rows=7 width=24) (actual tim
e=0.108..0.211 rows=273 loops=1)
                Recheck Cond: (('3765306.5850810907, -195178.73765543592, 5059147.49054945
9),(3865306.3290241663, -95178.99371236033, 5159147.234492534)::cube @> (ll_to_earth(latitude, longitude))
)::cube)
                Filter: ((work_status)::text = 'available'::text)
                Rows Removed by Filter: 80
                Heap Blocks: exact=82
                -> Bitmap Index Scan on idx_driver_location_gist  (cost=0.00..4.22 rows=9
width=0) (actual time=0.095..0.095 rows=353 loops=1)
                    Index Cond: ((ll_to_earth(latitude, longitude))::cube <@ '(3765306.5
850810907, -195178.73765543592, 5059147.490549459),(3865306.3290241663, -95178.99371236033, 5159147.2344925
34)::cube)
                    -> Index Scan using appuser_pkey on appuser u  (cost=0.29..1.19 rows=1 width=22) (actual ti
me=0.002..0.002 rows=1 loops=210)
                        Index Cond: (user_id = v.driver_id)
Planning Time: 0.721 ms
Execution Time: 2.787 ms
(23 rows)
```

5.3.2 Driver Performance Statistics

Requirement: Identify the top 10 drivers with the highest number of completed trips in the current month with average rating higher than 4.5

Constraints: trip.status = 'complete', time in current month, feedback.rating > 4.5

Data scale: ~8.900 active drivers, ~120.000 trips, ~56.000 feedbacks.

Approach:

- (1) INNER JOIN + GROUP BY
- (2) CTE
- (3) SUBQUERY + INDEX

(1) - INNER JOIN + GROUP BY	(2) – CTE	(3) – SUBQUERRY + INDEX
<pre> SELECT u.user_id, u.name, COUNT(t.trip_id) AS completed_trips, AVG(f.rating) AS avg_rating FROM appuser u INNER JOIN driver d ON u.user_id = d.user_id INNER JOIN trip t ON d.user_id = t.driver_id INNER JOIN feedback f ON t.trip_id = f.trip_id WHERE t.status = 'completed' AND EXTRACT(MONTH FROM t.end_time) = EXTRACT(MONTH FROM CURRENT_DATE) AND EXTRACT(YEAR FROM t.end_time) = EXTRACT(YEAR FROM CURRENT_DATE) GROUP BY u.user_id, u.name HAVING AVG(f.rating) > 4.5 ORDER BY completed_trips DESC LIMIT 10; </pre>	<pre> WITH MonthlyPerformance AS (SELECT t.driver_id, COUNT(t.trip_id) AS total_completed, AVG(f.rating) AS average_rating FROM trip t INNER JOIN feedback f ON t.trip_id = f.trip_id WHERE t.status = 'completed' AND EXTRACT(MONTH FROM t.end_time) = EXTRACT(MONTH FROM CURRENT_DATE) AND EXTRACT(YEAR FROM t.end_time) = EXTRACT(YEAR FROM CURRENT_DATE) GROUP BY t.driver_id HAVING AVG(f.rating) > 4.5) SELECT p.driver_id, u.name, p.completed_trips, p.avg_rating FROM MonthlyPerformance p INNER JOIN appuser u ON p.driver_id = u.user_id ORDER BY p.completed_trips DESC LIMIT 10; </pre>	<pre> CREATE INDEX idx_trip_perf ON trip (status, end_time, driver_id); CREATE INDEX idx_feedback_rating ON feedback (trip_id, rating); SELECT u.user_id, u.name, stats.completed_count, stats.avg_rating FROM appuser u INNER JOIN (SELECT t.driver_id, COUNT(*) AS completed_count, AVG(f.rating) AS avg_rating FROM trip t INNER JOIN feedback f ON t.trip_id = f.trip_id WHERE t.status = 'completed' AND EXTRACT(MONTH FROM t.end_time) = EXTRACT(MONTH FROM CURRENT_DATE) AND EXTRACT(YEAR FROM t.end_time) = EXTRACT(YEAR FROM CURRENT_DATE) GROUP BY t.driver_id) AS stats ON u.user_id = stats.driver_id WHERE stats.avg_rating > 4.5 ORDER BY stats.completed_count DESC LIMIT 10; </pre>
<pre> Planning Time: 5.517 ms Execution Time: 0.561 ms </pre>	<pre> Planning Time: 0.262 ms Execution Time: 0.052 ms (21 rows) </pre>	<pre> Planning Time: 0.519 ms Execution Time: 0.088 ms </pre>
→ It connects all tables first and filters later. This creates a large amount of data in memory before counting, which uses more resources.	→ It calculates trips and ratings in a separate small step first. It only joins the appuser table at the very end, making the process very efficient.	→ It uses a subquery to find driver IDs from the Index first. It avoids reading the main tables until it is necessary, which saves a lot of time as data grows.

5.3.3 Driver Rest Rule Checking

Requirement: Detect drivers who exceed 10 completed trips within a 3-hour window

Constraints: Time-ordered analysis, High query frequency

Data Scale: ~8,900 active drivers, ~120,000 trips

Query:

```

CREATE OR REPLACE FUNCTION func_check_rest()
RETURNS TRIGGER AS $$
DECLARE
  v_start_time TIMESTAMP;
  v_is_exceeded BOOLEAN;
BEGIN
  IF NEW.status = 'completed'
  AND (OLD.status IS NULL OR OLD.status != 'completed') THEN
    v_start_time := NEW.end_time - INTERVAL '3 hours';
    SELECT EXISTS (
      SELECT 1
      FROM trip
      WHERE driver_id = NEW.driver_id

```

```

        AND status = 'completed'
        AND end_time IS NOT NULL
        AND end_time >= v_start_time
    OFFSET 10 LIMIT 1
) INTO v_is_exceeded;

IF v_is_exceeded THEN
    UPDATE driver
    SET work_status = 'unavailable'
    WHERE user_id = NEW.driver_id
    AND work_status != 'unavailable';
END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;
-- Create trigger:
CREATE TRIGGER trg_check_driver_rest
AFTER UPDATE OF status
ON trip
FOR EACH ROW
WHEN (NEW.status = 'completed')
EXECUTE FUNCTION func_check_rest();

-- Check trigger:
BEGIN;

SELECT user_id, work_status
FROM driver
WHERE user_id = 4;
UPDATE trip
SET status = 'completed',
    end_time = NOW() - INTERVAL '30 minutes'
WHERE driver_id = 4
    AND trip_id IN (
        SELECT trip_id
        FROM trip
        WHERE driver_id = 4
        LIMIT 11
    );
SELECT user_id, work_status
FROM driver
WHERE user_id = 4;
ROLLBACK;

```

Design Rationale:

- **Automatic:** The trigger ensures the “10 trips per 3 hours” rule is checked immediately when a trip is completed
- **Data consistency and reliability:** AFTER UPDATE trigger guarantees the check is executed only after the trip record is successfully updated
- **Encapsulation:** Core rule logic is centralized inside a function

Performance Optimization

- **EXISTS with OFFSET/LIMIT instead of COUNT(*):** Stops scanning as soon as the 11th trip is found
- **Isolated time computation:** The 3-hour threshold is computed once and stored in a variable

- **Reduced trigger execution overhead:** The trigger is activated only when the trip status transitions to Completed

5.3.4 Speed Fraud Detection

Requirement: Detect trips with **abnormally high average speed** indicating potential fraud

Constraints: Accurate time-based calculation, High update frequency on Trip table, Minimal impact on core transaction performance

Data Scope: Trip data with start_time, end_time, and status, Distance retrieved from Trip_Request table

Query:

```
CREATE OR REPLACE FUNCTION func_check_speed_fraud()
RETURNS TRIGGER AS $$
DECLARE
    v_distance_km DECIMAL;
    v_duration_hours DECIMAL;
    v_avg_speed DECIMAL;
    v_threshold_speed CONSTANT INT := 120; -- km/h
BEGIN
    IF NEW.status = 'completed'
    AND (OLD.status IS NULL OR OLD.status != 'completed')
    AND NEW.start_time IS NOT NULL
    AND NEW.end_time IS NOT NULL THEN

        -- Calculate distance using Haversine formula
        SELECT
            6371 * 2 * ASIN(
                SQRT(
                    POWER(SIN(RADIANS(tr.drop_latitude - tr.pickup_latitude)
/ 2), 2) +
                    COS(RADIANS(tr.pickup_latitude)) *
                    COS(RADIANS(tr.drop_latitude)) *
                    POWER(SIN(RADIANS(tr.drop_longitude -
tr.pickup_longitude) / 2), 2)
                )
            )
        INTO v_distance_km
        FROM trip_request tr
        WHERE tr.trip_request_id = NEW.trip_request_id;
        v_duration_hours :=
            EXTRACT(EPOCH FROM (NEW.end_time - NEW.start_time)) / 3600;
        IF v_duration_hours > 0 AND v_distance_km IS NOT NULL THEN
            v_avg_speed := v_distance_km / v_duration_hours;
            IF v_avg_speed > v_threshold_speed THEN
                INSERT INTO fraud_logs (
                    driver_id, trip_id, reason, details, created_at
                )
                VALUES (
                    NEW.driver_id,
                    NEW.trip_id,
                    'Abnormal speed',
                    'Speed=' || ROUND(v_avg_speed,2) ||
                    ' km/h, Distance=' || ROUND(v_distance_km,2) || ' km',
                    NOW()
                );
            END IF;
        END IF;
    END IF;

    RETURN NEW;
```

```

END;
$$ LANGUAGE plpgsql;
-- Create trigger:
CREATE TRIGGER trg_speed_fraud_check
AFTER UPDATE OF status
ON trip
FOR EACH ROW
WHEN (NEW.status = 'completed')
EXECUTE FUNCTION func_check_speed_fraud();

--Check trigger:
BEGIN;

UPDATE trip
SET status = 'completed',
    start_time = NOW() - INTERVAL '5 minutes',
    end_time = NOW()
WHERE trip_id = 4;

ROLLBACK;

```

Design Rationale:

- **Data integrity and consistency**
- **Reduced backend workload**

Performance Optimization:

- **AFTER UPDATE trigger for accurate data:** Speed calculation is performed only after end_time and status are fully persisted
- **Isolated calculations:** Trip duration and distance are computed once and stored in variables
- **Trigger filtering to reduce overhead**
 - AFTER UPDATE OF status ensures the trigger fires only when status changes
 - Prevents unnecessary executions during frequent GPS or metadata updates

5.3.5 Periodic Vehicle Maintenance Check

Requirement: *List active vehicles whose total traveled distance exceeds the maintenance threshold.*

Business Logic: Aggregate traveled distance from trip_request, Vehicles exceeding 5,000 km are flagged for maintenance.

Query:

```

SELECT v.vehicle_id, v.license_plate
FROM vehicle v
WHERE v.status = 'active'
    AND EXISTS (
        SELECT 1
        FROM trip t
        JOIN trip_request tr ON t.trip_request_id = tr.trip_request_id
        WHERE t.vehicle_id = v.vehicle_id
        GROUP BY t.vehicle_id
        HAVING SUM(tr.distance) > 5000
    );

```

Optimization: Use EXISTS instead of IN

5.3.6 Average Fare Range Analysis

Requirement: *List completed trips with fares between 5 and 10\$.*

Business Logic: Filter completed trips based on fare range to support financial metrics analysis.

Query:

```
SELECT trip_id, fare, end_time
FROM trip
WHERE fare BETWEEN 5 AND 10
      AND status = 'completed';
```

Optimization:

Use BETWEEN instead of separate >= and <= .

5.3.7 High-Performance Driver Identification

Requirement: Identify drivers with good performance who have no low ratings (1–2 stars) and are currently active.

Business Logic: Select drivers who have never received negative customer feedback while maintaining an active status.

Query:

```
SELECT d.user_id, u.name
FROM driver d
JOIN appuser u ON d.user_id = u.user_id
LEFT JOIN trip t ON d.user_id = t.driver_id
LEFT JOIN feedback f ON t.trip_id = f.trip_id AND f.rating <= 2
WHERE u.status = 'active'
      AND f.feedback_id IS NULL;
```

Optimization: Use LEFT JOIN ... IS NULL instead of NOT IN

5.3.8 Find Replacement Drivers in Case of Vehicle Failure

Requirement: Find available drivers near the incident location to take over a trip, excluding the current driver who reported the vehicle failure.

Business Logic: Filter drivers by active account status and available work status, while explicitly excluding the affected driver from the result set.

Query:

```
-- Optimized approach (Tip 21: positive logic / ID range split)
SELECT d.user_id, d.latitude, d.longitude
FROM driver d
JOIN appuser u ON d.user_id = u.user_id
WHERE u.status = 'active'
      AND d.work_status = 'available'
      AND (d.user_id < 12345 OR d.user_id > 12345);
```

Optimization: Avoid `user_id != X`; use positive conditions (< or >) combined with status filtering to better leverage B-tree indexes and reduce scan range.

5.3.9 Service Type Vehicle Filtering

Requirement: Filter vehicles that match the selected service types (e.g., Car or Motorbike) and are ready for dispatch.

Business Logic: Select vehicles belonging to allowed service categories with active operational status.

Query:

```
SELECT vehicle_id, license_plate, type, driver_id
FROM vehicle
WHERE type IN ('car', 'motorbike')
      AND status = 'active';
```

Optimization: Use IN instead of multiple OR conditions to improve optimizer behavior and increase the chance of index usage.

5.3.10 Destination Autocomplete Search

Requirement: Provide destination suggestions as users type the first characters of a location name.

Business Logic: Retrieve distinct drop locations that start with the user's input prefix.

Query:

```
SELECT DISTINCT drop_location
FROM trip_request
WHERE drop_location LIKE 'Road %'
LIMIT 10;
```

Optimization: Avoid leading % in LIKE patterns. Using a fixed prefix allows the B-tree index on drop_location to be used, while LIMIT reduces data transfer.

5.3.11 Driver Performance Dashboard using Materialized View

Requirement: Provide administrators with a fast dashboard displaying driver KPIs, including total completed trips, total earnings, and average rating, without querying raw trip and feedback data in real time.

Business Logic: Pre-compute driver performance metrics by aggregating data from driver, trip, and feedback, then store the results in a materialized view for fast access.

Query:

```
CREATE MATERIALIZED VIEW mv_driver_kpi_dashboard AS
SELECT
    d.user_id,
    u.name,
    COUNT(t.trip_id) AS total_completed_trips,
    SUM(t.fare) AS total_earnings,
    ROUND(AVG(f.rating), 2) AS avg_rating,
    u.status AS account_status
FROM driver d
JOIN appuser u ON d.user_id = u.user_id
LEFT JOIN trip t ON d.user_id = t.driver_id AND t.status = 'completed'
LEFT JOIN feedback f ON t.trip_id = f.trip_id
GROUP BY d.user_id, u.name, u.status;

CREATE INDEX idx_mv_driver_id
ON mv_driver_kpi_dashboard(user_id);
```

```
SELECT *  
FROM mv_driver_kpi_dashboard  
WHERE total_completed_trips > 50  
ORDER BY avg_rating DESC  
LIMIT 20;
```

Optimization: Use a Materialized View to pre-compute and persist aggregated KPI data, reducing expensive JOIN and GROUP BY operations on large datasets and enabling millisecond-level dashboard queries.

5.4 Enterprise Queries

5.4.1

Insert new male appuser with user_id=1000001, name='Nguyen Van A', phone='0987654321', email='nguyenvana@example.com', role= 'driver', join_date is current_date and status is active

```
insert into appuser (user_id, name, phone, email, gender, role, join_date, status)
values (1000001, 'Nguyen Van A', '0987654321', 'nguyenvana@example.com', 1,
'driver', current_date, 'active');
```

5.4.2

Find the top 10 drivers by total revenue from completed trips. For each driver, show driver ID, driver name, number of completed trips, total revenue, and average rating (if any).

Only include drivers with at least 5 completed trips.

Native:

```
select d.user_id,
au.name as driver_name,
count(t.trip_id) as completed_trips,
sum(t.fare) as total_revenue,
avg(f.rating) as avg_rating
from trip t
join driver d on d.user_id = t.driver_id
join appuser au on au.user_id = d.user_id
left join feedback f on f.trip_id = t.trip_id
where t.status = 'completed'
group by d.user_id, au.name
having count(t.trip_id) >= 5
order by total_revenue desc
limit 10;
```

	QUERY PLAN
	text
24	-> Hash (cost=1213.00..1213.00 rows=50000 width=22) (actual time=9.210..9.211 rows=50000 loo...
25	Buckets: 65536 Batches: 1 Memory Usage: 3306kB
26	-> Seq Scan on appuser au (cost=0.00..1213.00 rows=50000 width=22) (actual time=0.009..3.8...
27	-> Hash (cost=966.09..966.09 rows=55909 width=10) (actual time=7.591..7.592 rows=55909 loops=1)
28	Buckets: 65536 Batches: 1 Memory Usage: 3133kB
29	-> Seq Scan on feedback f (cost=0.00..966.09 rows=55909 width=10) (actual time=0.019..3.351 ro...
30	Planning Time: 0.768 ms
31	Execution Time: 102.269 ms

- It aggregates over all completed trips with joins, which can be heavy when tables grow.

- The left join feedback can inflate rows if a trip can have multiple feedback rows (not guaranteed unique), which can distort count() or sum().

Optimized:

```
create index idx_trip_completed_driverid on trip (driver_id) where
status = 'completed';
create index idx_feedback_tripid on feedback (trip_id);
explain analyze
with driver_have5_completedtrip as (
    select
        t.driver_id,
        count(*) as completed_trips,
        sum(t.fare) as total_revenue
    from trip t
    where t.status = 'completed'
    group by t.driver_id
    having count(*) >= 5
),
avgfeedback_on_trip as (
    select
        f.trip_id,
        avg(f.rating) as avg_trip_rating
    from feedback f
    group by f.trip_id
),
avgfeedback_on_driver as (
    select
        t.driver_id,
        avg(fbt.avg_trip_rating) as avg_rating
    from trip t
    join avgfeedback_on_trip fbt
    on fbt.trip_id = t.trip_id
    where t.status = 'completed'
    group by t.driver_id
)
select
    d.user_id,
    au.name as driver_name,
    ta.completed_trips,
    ta.total_revenue,
    ra.avg_rating
from driver_have5_completedtrip ta
join driver d
on d.user_id = ta.driver_id
```

```

join appuser au
on au.user_id = d.user_id
left join avgfeedback_on_driver ra
on ra.driver_id = ta.driver_id
order by ta.total_revenue desc
limit 10;

```

	QUERY PLAN text	
36	-> Index Scan using idx_feedback_tripid on feedback f (cost=0.29..1868.93 rows=55909 width=10) (actual time=0.014.....	
37	-> Hash (cost=2470.00..2470.00 rows=60153 width=16) (actual time=10.807..10.808 rows=60026 loops=1)	
38	Buckets: 65536 Batches: 1 Memory Usage: 3326kB	
39	-> Seq Scan on trip t_1 (cost=0.00..2470.00 rows=60153 width=16) (actual time=0.017..6.880 rows=60026 loops=1)	
40	Filter: ((status)::text = 'completed'::text)	
41	Rows Removed by Filter: 39974	
42	Planning Time: 0.688 ms	
43	Execution Time: 84.648 ms	

- driver_have5_completedtrip groups trip rows by driver_id and filters having count(*) >= 5 early.

That means the later joins to driver and appuser happen on a much smaller result set, instead of on millions of trip rows.

- Instead of joining feedback directly to trip, pre-aggregate:

avgfeedback_on_trip: one row per trip_id

avgfeedback_on_driver: average rating per driver from those per-trip ratings

This makes the final aggregation correct and stable, and reduces join work.

- The index on trip(driver_id) where t.status = 'completed' is a partial index stores only those rows, so driver_have5_completedtrip and avgfeedback_on_driver can scan far fewer tuples and group faster.

- The index on feedback(trip_id) make both avgfeedback_on_trip and the join fbt.trip_id = t.trip_id benefit because trip_id is the exact join column. This reduces random I/O and speeds up hash/merge planning.

5.4.3

List drivers who are marked as available but have not updated their location recently. A driver is considered stale if driver.last_update < now() - interval '10 minutes'.

Return driver id, driver name, driver.work_status, driver.last_update, latitude, longitude, and the number of active trips (status in ('assigned','in_progress')). Sort by driver.last_update ascending (oldest first).

Native:

```

select
d.user_id as driver_id,
u.name as driver_name,

```

```

d.work_status,
d.last_update,
d.latitude,
d.longitude,
count(t.trip_id) as active_trips
from driver d
join appuser u
on u.user_id = d.user_id
left join trip t
on t.driver_id = d.user_id
and t.status in ('assigned','in_progress')
where d.work_status = 'available'
and d.last_update < now() - interval '10 minutes'
group by
d.user_id,    u.name,    d.work_status,    d.last_update,    d.latitude,
d.longitude
order by d.last_update asc;

```

	QUERY PLAN	
	text	
11	Rows Removed by Filter: 69984	
12	-> Hash (cost=1695.52..1695.52 rows=7133 width=56) (actual time=7.282..7.285 rows=7134 loops=1)	
13	Buckets: 8192 Batches: 1 Memory Usage: 731kB	
14	-> Hash Join (cost=351.26..1695.52 rows=7133 width=56) (actual time=1.642..6.486 rows=7134 l...	
15	Hash Cond: (u.user_id = d.user_id)	
16	-> Seq Scan on appuser u (cost=0.00..1213.00 rows=50000 width=22) (actual time=0.006..1.58...	
17	-> Hash (cost=262.10..262.10 rows=7133 width=42) (actual time=1.613..1.614 rows=7134 loo...	
18	Buckets: 8192 Batches: 1 Memory Usage: 632kB	
19	-> Seq Scan on driver d (cost=0.00..262.10 rows=7133 width=42) (actual time=0.004..0.970...	
20	Filter: (((work_status)::text = 'available'::text) AND (last_update < (now() - '00:10:00'::inter...	
21	Rows Removed by Filter: 1771	
22	Planning Time: 0.223 ms	
23	Execution Time: 27.747 ms	

- The left join trip can produce many rows per driver, then group by must re-aggregate, which becomes expensive as trip grows.
- Postgres scan a large part of trip to find active trips, making the join + aggregation heavy.

Optimized:

```

create index on trip (driver_id)
where status in ('assigned','in_progress');
create index idx_driver_available_last_update on driver (last_update)
where work_status = 'available';
with stale_drivers as (
select

```

```

d.user_id,
u.name,
d.work_status,
d.last_update,
d.latitude,
d.longitude
from driver d
join appuser u
on u.user_id = d.user_id
where d.work_status = 'available'
and d.last_update < now() - interval '10 minutes'
),
active_trip_counts as (
select
t.driver_id,
count(*) as active_trips
from trip t
where t.status in ('assigned','in_progress')
group by t.driver_id
)
select
sd.user_id as driver_id,
sd.name as driver_name,
sd.work_status,
sd.last_update,
sd.latitude,
sd.longitude,
coalesce(atc.active_trips, 0) as active_trips
from stale_drivers sd
left join active_trip_counts atc
on atc.driver_id = sd.user_id
order by sd.last_update asc;

```

	QUERY PLAN
	text
6	Hash Cond: (u.user_id = d.user_id)
7	-> Seq Scan on appuser u (cost=0.00..1213.00 rows=50000 width=22) (actual time=0.009..1.577 rows=50000 loops=1)
8	-> Hash (cost=262.10..262.10 rows=7133 width=42) (actual time=1.762..1.762 rows=7134 loops=1)
9	Buckets: 8192 Batches: 1 Memory Usage: 632kB
10	-> Seq Scan on driver d (cost=0.00..262.10 rows=7133 width=42) (actual time=0.007..0.977 rows=7134 loops=1)
11	Filter: (((work_status)::text = 'available'::text) AND (last_update < (now() - '00:10:00'::interval)))
12	Rows Removed by Filter: 1771
13	-> Aggregate (cost=4.35..4.36 rows=1 width=4) (actual time=0.001..0.001 rows=1 loops=7134)
14	-> Index Only Scan using trip_driver_id_idx on trip t (cost=0.29..4.34 rows=3 width=0) (actual time=0.000..0.001 rows=3 lo...
15	Index Cond: (driver_id = d.user_id)
16	Heap Fetches: 0
17	Planning Time: 0.484 ms
18	Execution Time: 16.116 ms

- Filters stale available drivers first, so trip counting runs only for a smaller set of drivers.
- Avoids a large join + group by; counts active trips per driver directly.
- Uses partial indexes to speed up both the stale-driver filter and the active-trip lookup.

5.4.4

Suspend drivers whose trips were canceled at least 5 times in the last 30 days.

Update the driver's account status to suspended.

Native:

```
update appuser au
set status = 'suspended'
where au.role = 'driver'
and au.status <> 'suspended'
and (
select count(*)
from trip t
join trip_request tr
on tr.trip_request_id = t.trip_request_id
where t.driver_id = au.user_id
and t.status = 'canceled'
and tr.request_time >= current_date - interval '30 days'
) >= 5;
```


	QUERY PLAN	
	text	
4	Rows Removed by Filter: 50000	
5	SubPlan 1	
6	-> Aggregate (cost=2728.32..2728.33 rows=1 width=8) (actual time=3.805..3.805 rows=1 loops=9887)	
7	-> Nested Loop (cost=0.29..2728.32 rows=1 width=0) (actual time=3.797..3.803 rows=0 loops=9887)	
8	-> Seq Scan on trip t (cost=0.00..2720.00 rows=1 width=8) (actual time=2.424..3.788 rows=1 loops=9887)	
9	Filter: ((driver_id = au.user_id) AND ((status)::text = 'canceled'::text))	
10	Rows Removed by Filter: 99999	
11	-> Index Scan using trip_request_pkey on trip_request tr (cost=0.29..8.32 rows=1 width=8) (actual time=0.006..0.006 rows=1 loops=1)	
12	Index Cond: (trip_request_id = t.trip_request_id)	
13	Filter: (request_time >= (CURRENT_DATE - '30 days'::interval))	
14	Rows Removed by Filter: 1	
15	Planning Time: 0.389 ms	
16	Execution Time: 37648.831 ms	

- The correlated subquery is evaluated per driver row, repeatedly scanning/joining trip + trip_request.

- With many drivers and many trips, this becomes expensive because the database does lots of repeated work.

Optimized:

```
create index idx_trip_canceled_driver_trid on trip (driver_id,
trip_request_id)
where status = 'canceled';
create index idx_trip_request_request_time_trid on trip_request
(request_time, trip_request_id);
with canceled_30d as (
select
t.driver_id
from trip t
join trip_request tr
on tr.trip_request_id = t.trip_request_id
where t.status = 'canceled'
and tr.request_time >= now() - interval '30 days'
group by t.driver_id
having count(*) >= 5
)
update appuser au
set status = 'suspended'
from canceled_30d c
where au.user_id = c.driver_id
and au.role = 'driver'
and au.status <> 'suspended';
```

	QUERY PLAN
	text
10	Hash Cond: (t.trip_request_id = tr.trip_request_id)
11	-> Index Only Scan using idx_trip_canceled_driver_trid on trip t (cost=0.29..310.28 rows=9733 width=16) (actual time=0.026..0.649 rows=9958 loop...
12	Heap Fetches: 0
13	-> Hash (cost=183.53..183.53 rows=5435 width=8) (actual time=0.687..0.688 rows=5319 loops=1)
14	Buckets: 8192 Batches: 1 Memory Usage: 272kB
15	-> Index Only Scan using idx_trip_request_request_time_trid on trip_request tr (cost=0.42..183.53 rows=5435 width=8) (actual time=0.044..0.419...
16	Index Cond: (request_time >= (now() - '30 days'::interval))
17	Heap Fetches: 0
18	-> Index Scan using appuser_pkey on appuser au (cost=0.29..6.34 rows=1 width=14) (never executed)
19	Index Cond: (user_id = c.driver_id)
20	Filter: (((status)::text <> 'suspended'::text) AND ((role)::text = 'driver'::text))
21	Planning Time: 1.727 ms
22	Execution Time: 1.946 ms

- The CTE computes the “drivers with ≥ 5 canceled trips in last 30 days” once, then the update touches only matching drivers.
- idx_trip_canceled_driver_trid is a partial index that keeps only canceled trips, so the planner can find canceled trips per driver with far fewer rows.
- idx_trip_request_request_time_trid supports the 30-day time filter and the join key, reducing work when filtering recent requests and joining to trips.

5.4.5

Delete a vehicle from the system given a vehicle_id=2001. Only delete it if the vehicle is currently in maintenance, has no assigned driver, and has never been used in any trip.

```
delete from vehicle v
where v.vehicle_id = 2001
and v.status = 'maintenance'
and v.driver_id is null
and not exists (
select 1
from trip t
where t.vehicle_id = v.vehicle_id
);
```

- This sql command is optimized because Postgre will find the vehicle by vehicle_id, which is already indexed (pk)

5.4.6

For each vehicle, compute:

- total number of trips (all statuses),
- revenue from completed trips only,
- the latest trip end time (most recent activity).

Include vehicles even if they have never been used in any trip. Sort by completed-trip revenue descending.

Native:

```

select
v.vehicle_id,
v.license_plate,
count(t.trip_id) as total_trips_all_statuses,
coalesce(sum(
case when t.status = 'completed'
then t.fare else 0
end), 0) as completed_trip_revenue,
max(t.end_time) as latest_trip_end_time
from vehicle v
left join trip t
on t.vehicle_id = v.vehicle_id
group by
v.vehicle_id,
v.license_plate
order by
completed_trip_revenue desc;

```

	QUERY PLAN	
	text	
2	Sort Key: (COALESCE(sum(CASE WHEN ((t.status)::text = 'completed'::text) THEN t.fare ELSE '0'::numeric END)),...	
3	Sort Method: quicksort Memory: 941kB	
4	-> HashAggregate (cost=4014.98..4126.30 rows=8905 width=65) (actual time=36.484..38.864 rows=8905 loo...	
5	Group Key: v.vehicle_id	
6	Batches: 1 Memory Usage: 4497kB	
7	-> Hash Right Join (cost=282.36..2764.98 rows=100000 width=49) (actual time=1.408..18.942 rows=1000...	
8	Hash Cond: (t.vehicle_id = v.vehicle_id)	
9	-> Seq Scan on trip t (cost=0.00..2220.00 rows=100000 width=40) (actual time=0.016..3.404 rows=100...	
10	-> Hash (cost=171.05..171.05 rows=8905 width=17) (actual time=1.325..1.326 rows=8905 loops=1)	
11	Buckets: 16384 Batches: 1 Memory Usage: 555kB	
12	-> Seq Scan on vehicle v (cost=0.00..171.05 rows=8905 width=17) (actual time=0.010..0.541 rows=...	
13	Planning Time: 0.312 ms	
14	Execution Time: 41.981 ms	

- It scans and aggregates all trips per vehicle even though revenue only needs completed trips.
- With a large trip table, the join + group by can be heavy.
- order by completed_trip_revenue desc forces sorting the full aggregated result.

Optimized:

```

create index idx_trip_vehicle_id on trip (vehicle_id);
create index idx_trip_vehicle_completed on trip (vehicle_id)
where status = 'completed';
create index idx_trip_vehicle_end_time on trip (vehicle_id, end_time
desc);

```

```

with trip_by_vehicle as (
select
t.vehicle_id,
count(*) as total_trips_all_statuses,
sum(t.fare) filter (where t.status = 'completed') as
completed_trip_revenue,
max(t.end_time) as latest_trip_end_time
from trip t
group by t.vehicle_id
)
select
v.vehicle_id,
v.license_plate,
coalesce(tb.total_trips_all_statuses, 0) as total_trips_all_statuses,
coalesce(tb.completed_trip_revenue, 0) as completed_trip_revenue,
tb.latest_trip_end_time
from vehicle v
left join trip_by_vehicle tb
on tb.vehicle_id = v.vehicle_id
order by
completed_trip_revenue desc;

```

	QUERY PLAN	
	text	
3	Sort Method: quicksort Memory: 941kB	
4	-> Hash Left Join (cost=3777.96..3972.40 rows=8905 width=65) (actual time=25.680..28.389 rows=8905 loops=...	
5	Hash Cond: (v.vehicle_id = tb.vehicle_id)	
6	-> Seq Scan on vehicle v (cost=0.00..171.05 rows=8905 width=17) (actual time=0.014..0.432 rows=8905 loo...	
7	-> Hash (cost=3667.98..3667.98 rows=8799 width=56) (actual time=25.588..25.590 rows=8905 loops=1)	
8	Buckets: 16384 Batches: 1 Memory Usage: 685kB	
9	-> Subquery Scan on tb (cost=3470.00..3667.98 rows=8799 width=56) (actual time=21.207..24.537 rows=...	
10	-> HashAggregate (cost=3470.00..3579.99 rows=8799 width=56) (actual time=21.205..23.922 rows=8...	
11	Group Key: t.vehicle_id	
12	Batches: 1 Memory Usage: 4241kB	
13	-> Seq Scan on trip t (cost=0.00..2220.00 rows=100000 width=32) (actual time=0.005..3.433 rows=...	
14	Planning Time: 0.183 ms	
15	Execution Time: 33.266 ms	

- The query aggregates trip once by vehicle_id; idx_trip_vehicle_id helps the planner read/group trips by vehicle efficiently.
- Revenue uses filter (where status = 'completed'); the partial index idx_trip_vehicle_completed can speed access to completed rows.

- max(end_time) benefits from idx_trip_vehicle_end_time (vehicle_id, end_time desc) because the planner can find the latest end time per vehicle more cheaply than scanning all end times.
- Joining the aggregated result back to vehicle keeps vehicles with zero trips (required), while reducing join volume versus joining all trip rows first.

5.4.7

Create a view named v_vehicle_rating_30d that returns one row per vehicle with:

- vehicle_id
- license_plate
- completed_trips_30d: number of trips with trip.status = 'completed' in the last 30 days
- avg_rating_30d: average feedback.rating of those completed trips in the last 30 days

Important requirements:

- Include all vehicles, even if they have zero trips or no feedback.
- For unused vehicles: completed_trips_30d = 0, avg_rating_30d = null.

Native:

```
create or replace view v_vehicle_rating_30d as
with completed_30d as (
select
t.trip_id,
t.vehicle_id
from trip t
where t.status = 'completed'
and date(t.end_time) >= current_date - interval '30 days'
),
agg as (
select
c.vehicle_id,
count(*) as completed_trips_30d,
avg(f.rating) as avg_rating_30d
from completed_30d c
left join feedback f on f.trip_id = c.trip_id
group by c.vehicle_id
)
select
v.vehicle_id,
v.license_plate,
coalesce(a.completed_trips_30d, 0) as completed_trips_30d,
a.avg_rating_30d
```

```

from vehicle v
left join agg a on a.vehicle_id = v.vehicle_id;

```

- Cost of loading data:

	QUERY PLAN	
	text	
/	-> HashAggregate (cost=4983.88..5085.22 rows=810/ width=48) (actual time=65.20/..66.83...	
8	Group Key: t.vehicle_id	
9	Batches: 1 Memory Usage: 913kB	
10	-> Hash Right Join (cost=3720.64..4833.50 rows=20051 width=10) (actual time=61.534....	
11	Hash Cond: (f.trip_id = t.trip_id)	
12	-> Seq Scan on feedback f (cost=0.00..966.09 rows=55909 width=10) (actual time=0.4...	
13	-> Hash (cost=3470.00..3470.00 rows=20051 width=16) (actual time=44.528..44.530 r...	
14	Buckets: 32768 Batches: 1 Memory Usage: 414kB	
15	-> Seq Scan on trip t (cost=0.00..3470.00 rows=20051 width=16) (actual time=40.6...	
16	Filter: (((status)::text = 'completed'::text) AND (date(end_time) >= (CURRENT_DA...	
17	Rows Removed by Filter: 96641	
18	Planning Time: 17.771 ms	
19	Execution Time: 74.352 ms	

- The filter uses `date(t.end_time)`, which applies a function to the column and often prevents PostgreSQL from using a normal b-tree index on `end_time` efficiently.

- PostgreSQL may do a sequential scan on trip to find the last-30-days completed trips.

Optimized:

```

create index idx_trip_vehicle_completed_end_time on trip (vehicle_id, end_time
desc)
where status = 'completed';
create index if not exists idx_feedback_trip_id on feedback (trip_id);
create or replace view v_vehicle_rating_30d as
with completed_30d as (
select
t.trip_id,
t.vehicle_id
from trip t
where t.status = 'completed'
and date(t.end_time) >= current_date - interval '30 days'
),
agg as (
select
c.vehicle_id,
count(*) as completed_trips_30d,
avg(f.rating) as avg_rating_30d

```

```

from completed_30d c
left join feedback f on f.trip_id = c.trip_id
group by c.vehicle_id
)
select
v.vehicle_id,
v.license_plate,
coalesce(a.completed_trips_30d, 0) as completed_trips_30d,
a.avg_rating_30d
from vehicle v
left join agg a on a.vehicle_id = v.vehicle_id;

```

- Cost of loading data:

	QUERY PLAN	
	text	
/	-> HashAggregate (cost=4983.88..5085.22 rows=810 / width=48) (actual time=50.892..54.04...	
8	Group Key: t.vehicle_id	
9	Batches: 1 Memory Usage: 913kB	
10	-> Hash Right Join (cost=3720.64..4833.50 rows=20051 width=10) (actual time=46.517.....	
11	Hash Cond: (f.trip_id = t.trip_id)	
12	-> Seq Scan on feedback f (cost=0.00..966.09 rows=55909 width=10) (actual time=0.0...	
13	-> Hash (cost=3470.00..3470.00 rows=20051 width=16) (actual time=33.633..33.635 r...	
14	Buckets: 32768 Batches: 1 Memory Usage: 414kB	
15	-> Seq Scan on trip t (cost=0.00..3470.00 rows=20051 width=16) (actual time=30.5...	
16	Filter: (((status)::text = 'completed'::text) AND (date(end_time) >= (CURRENT_DA...	
17	Rows Removed by Filter: 96641	
18	Planning Time: 0.523 ms	
19	Execution Time: 63.363 ms	

- idx_trip_completed_end_time stores only completed trips and supports the 30-day filter, so the planner can avoid scanning all trip rows.

- idx_feedback_trip_id makes the join feedback.trip_id = trip.trip_id fast because index lookup instead of scanning feedback.

- idx_trip_vehicle_id helps grouping / aggregating by vehicle_id when building agg.

5.4.8

The enterprise regularly receives customer ratings after each completed trip.

Some vehicles perform poorly and should be taken out of service to ensure service quality.

- Requirement

- A vehicle must be automatically marked as “maintenance” (status='maintenance') when both of the following conditions are met within the last 30 days:
 - The vehicle has completed at least 3 trips, and
 - The average customer rating of those trips is 2.5 or lower.
- Operational rules
 - The check should happen immediately when a new rating is submitted or an existing rating is updated.
 - Only the vehicle related to that trip should be evaluated.
 - Vehicles that do not meet the conditions must remain unchanged.

Native:

```
create or replace function tg_vehicle_maintenance_native() returns
trigger
as $$
declare
v_vehicle_id bigint;
v_completed_cnt int;
v_avg_rating numeric;
begin
select t.vehicle_id
into v_vehicle_id
from trip t
where t.trip_id = new.trip_id;
select
count(*) as completed_cnt,
avg(f.rating) as avg_rating
into
v_completed_cnt,
v_avg_rating
from trip t
join feedback f
on f.trip_id = t.trip_id
where t.vehicle_id = v_vehicle_id
and t.status = 'completed'
and t.end_time >= now() - interval '30 days';
if v_completed_cnt >= 3 and v_avg_rating <= 2.5 then
update vehicle
set status = 'maintenance'
where vehicle_id = v_vehicle_id
and status <> 'maintenance';
end if;
return new;
```



```

end;
$$
language plpgsql;

create or replace trigger trg_vehicle_maintenance_native
after insert or update of rating on feedback
for each row
execute function tg_vehicle_maintenance_native();

```

	QUERY PLAN	
	text	
1	Update on feedback (cost=0.29..8.31 rows=0 width=0) (actual time=0.021..0.022 rows=0 loops=1)	
2	-> Index Scan using idx_feedback_trip_id on feedback (cost=0.29..8.31 rows=1 width=8) (actual time=0.019..0.019 rows=0 loop...	
3	Index Cond: (trip_id = 123)	
4	Planning Time: 0.122 ms	
5	Execution Time: 0.054 ms	

- Every feedback insert/update recomputes the 30-day aggregates by scanning many trip rows for that vehicle.
- The join trip -> feedback and the 30-day filter can degrade to sequential scans.

Optimized:

```

create index idx_trip_vehicle_completed_end_time on trip (vehicle_id,
end_time desc)
where status = 'completed';
create index idx_feedback_trip_id on feedback (trip_id);

```

```

create or replace function tg_vehicle_maintenance() returns trigger
as $$
declare
v_vehicle_id bigint;
v_completed_cnt int;
v_avg_rating numeric;
begin
select t.vehicle_id
into v_vehicle_id
from trip t
where t.trip_id = new.trip_id;

```

```

select
count(*) as completed_cnt,
avg(f.rating) as avg_rating
into
v_completed_cnt,
v_avg_rating
from trip t

```

```

left join feedback f
on f.trip_id = t.trip_id
where t.vehicle_id = v_vehicle_id
and t.status = 'completed'
and t.end_time >= now() - interval '30 days';

if v_completed_cnt >= 3 and v_avg_rating <= 2.5 then
update vehicle
set status = 'maintenance'
where vehicle_id = v_vehicle_id
and status <> 'maintenance';
end if;

return new;
end;
$$
language plpgsql;

create or replace trigger trg_vehicle_maintenance
after insert or update of rating on feedback
for each row
execute function tg_vehicle_maintenance();

```

	QUERY PLAN text	
1	Update on public.feedback (cost=0.29..8.31 rows=0 width=0) (actual time=0.020..0.020 rows=0 loops=1)	
2	Buffers: shared hit=2	
3	-> Index Scan using idx_feedback_trip_id on public.feedback (cost=0.29..8.31 rows=1 width=8) (actual time=0.018..0.018 rows=0 loop...	
4	Output: rating, ctid	
5	Index Cond: (feedback.trip_id = 123)	
6	Buffers: shared hit=2	
7	Planning Time: 0.117 ms	
8	Execution Time: 0.047 ms	

- idx_trip_vehicle_completed_end_time is a partial index that targets exactly completed trips for one vehicle in the last 30 days, reducing scanned rows dramatically.
- idx_feedback_trip_id makes joining feedback for those trips fast.
- The trigger evaluates only the vehicle tied to the changed feedback, keeping work localized and predictable.

5.4.9

For the last 30 days, generate a daily (base on end_time) finance report including:

- day

- `completed_trips`
- `gross_revenue` (total `trip.fare`) of completed trips
- `collected_revenue` (total `payment.amount`) where `payment.status='completed'` (only for completed trips)
- `missing_payment_cnt` = completed trips without payment
- `mismatch_cnt` = completed trips where payment completed but `payment.amount` `trip.fare`
- `collected_7d_sum` (total `collected_revenue` of previous 7 days end at `current_date`)

Sort by day ascending.

Native:

```
with daily as (
select
date(t.end_time) as day,
count(*) as completed_trips,
sum(t.fare) as gross_revenue,
coalesce(sum(p.amount) filter (where p.status = 'completed'), 0) as
collected_revenue,
count(*) filter (where p.trip_id is null) as missing_payment_cnt,
count(*) filter (where p.status = 'completed' and p.amount <> t.fare)
as mismatch_cnt
from trip t
left join payment p
on p.trip_id = t.trip_id
where t.status = 'completed'
and t.end_time >= current_date - interval '30 days'
and t.end_time < current_date + interval '1 day'
group by 1
)
select
d.day,
d.completed_trips,
d.gross_revenue,
d.collected_revenue,
d.missing_payment_cnt,
d.mismatch_cnt,
(
select coalesce(sum(p2.amount), 0)
from trip t2
```

```

join payment p2
on p2.trip_id = t2.trip_id
where t2.status = 'completed'
and p2.status = 'completed'
and date(t2.end_time) between d.day - 6 and d.day
) as collected_7d_sum
from daily d
order by d.day asc;

```

QUERY PLAN		text	
id	plan		
16	-> Aggregate	(cost=5125.72..5125.73 rows=1 width=32) (actual time=15.853..15.853 rows=1 loops=17)	
17	-> Hash Join	(cost=3723.76..5125.29 rows=172 width=6) (actual time=15.398..15.779 rows=1165 loops=17)	
18	Hash Cond:	(p2.trip_id = t2.trip_id)	
19	-> Seq Scan on payment p2	(cost=0.00..1251.33 rows=57215 width=14) (actual time=0.003..4.302 rows=57091 loops=17)	
20	Filter:	((status)::text = 'completed'::text)	
21	Rows Removed by Filter:	2935	
22	-> Hash	(cost=3720.00..3720.00 rows=301 width=8) (actual time=9.120..9.120 rows=1230 loops=17)	
23	Buckets:	2048 (originally 1024) Batches: 1 (originally 1) Memory Usage: 76kB	
24	-> Seq Scan on trip t2	(cost=0.00..3720.00 rows=301 width=8) (actual time=8.607..9.045 rows=1230 loops=17)	
25	Filter:	(((((status)::text = 'completed'::text) AND (date(end_time) <= d.day) AND (date(end_time) >= (d.day - 6))))	
26	Rows Removed by Filter:	98770	
27	Planning Time:	7.044 ms	
28	Execution Time:	301.112 ms	

- The collected_7d_sum is a correlated subquery: it re-scans trip + payment once per day row.
- The daily aggregates are computed once, but the 7-day sum ignores that work and recomputes revenue from base tables again.

Optimized:

```

create index idx_trip_completed_end_time on trip (end_time)
where status = 'completed';
create index idx_payment_completed_trip_id on payment (trip_id)
where status = 'completed';
with daily as (
select
date(t.end_time) as day,
count(*) as completed_trips,
sum(t.fare) as gross_revenue,
coalesce(sum(p.amount) filter (where p.status = 'completed'), 0) as
collected_revenue,
count(*) filter (where p.trip_id is null) as missing_payment_cnt,
count(*) filter (where p.status = 'completed' and p.amount <> t.fare)
as mismatch_cnt
from trip t
left join payment p
on p.trip_id = t.trip_id
where t.status = 'completed'

```

```

and t.end_time >= current_date - interval '30 days'
and t.end_time < current_date + interval '1 day'
group by day
),
collected_7d as (
select
d1.day,
coalesce(sum(d2.collected_revenue), 0) as collected_7d_sum
from daily d1
left join daily d2
on d2.day between d1.day - 6 and d1.day
group by d1.day
)
select
d.day,
d.completed_trips,
d.gross_revenue,
d.collected_revenue,
d.missing_payment_cnt,
d.mismatch_cnt,
c.collected_7d_sum
from daily d
join collected_7d c
on c.day = d.day
order by d.day asc;

```

	QUERY PLAN text	
19	Sort Key: c.day	
20	Sort Method: quicksort Memory: 25kB	
21	-> Subquery Scan on c (cost=208106.03..208110.53 rows=200 width=36) (actual time=0.041..0.045 rows=17 loops=1)	
22	-> HashAggregate (cost=208106.03..208108.53 rows=200 width=36) (actual time=0.041..0.044 rows=17 loops=1)	
23	Group Key: d1.day	
24	Batches: 1 Memory Usage: 40kB	
25	-> Nested Loop Left Join (cost=0.00..203986.74 rows=823859 width=36) (actual time=0.002..0.029 rows=98 loops=1)	
26	Join Filter: ((d2.day <= d1.day) AND (d2.day >= (d1.day - 6)))	
27	Rows Removed by Join Filter: 191	
28	-> CTE Scan on daily d1 (cost=0.00..54.46 rows=2723 width=4) (actual time=0.000..0.001 rows=17 loops=1)	
29	-> CTE Scan on daily d2 (cost=0.00..54.46 rows=2723 width=36) (actual time=0.000..0.001 rows=17 loops=1)	
30	Planning Time: 0.637 ms	
31	Execution Time: 7.728 ms	

- The 7-day sum is computed by self-joining the already-aggregated daily result, instead of re-scanning the big base tables per day.
- idx_trip_completed_end_time supports the range filter for the last 30 days on completed trips.

- idx_payment_completed_trip_id supports fast access to only completed payments during aggregation.

5.4.10

Create function fn_driver_earning(driver_id bigint, day1 date, day2 date) that returns the total earning of a driver for the period [day1, day2]

- Only include trips where trip.status = 'completed' and payment.status = 'completed'.

- A driver's earning from a trip equals trip.fare.

- Trips with missing payments or failed payments must not be counted.

Native:

```
create or replace function fn_driver_earning(p_driver_id bigint,
p_day1 date, p_day2 date) returns numeric
as
$$
declare
v_total numeric;
begin
select coalesce(sum(t.fare), 0)
into v_total
from trip t
join payment p
on p.trip_id = t.trip_id
where t.driver_id = p_driver_id
and t.status = 'completed'
and p.status = 'completed'
and t.end_time >= p_day1
and t.end_time < p_day2 + 1;

return v_total;
end;
$$
language plpgsql;
```

	QUERY PLAN	
	text	
1	Result (cost=0.00..0.26 rows=1 width=32) (actual time=5.582..5.583 rows=1 loops=...	
2	Planning Time: 0.018 ms	
3	Execution Time: 5.589 ms	

- It relies on default indexes only (PK/unique). With large data, filtering by driver_id + status + end_time can still scan many rows.

- payment is joined and then filtered by p.status='completed'; without a supporting index, PostgreSQL may read many payment rows before filtering.

Optimized:

```
create index idx_trip_driver_completed_end_time on trip (driver_id,
end_time)
where status = 'completed';
create index idx_payment_trip_completed on payment (trip_id)
where status = 'completed';
create or replace function fn_driver_earning(p_driver_id bigint,
p_day1 date, p_day2 date) returns numeric
as
$$
declare
res numeric;
begin
select coalesce(sum(t.fare), 0)
into res
from trip t
join payment p
on p.trip_id = t.trip_id
where t.driver_id = p_driver_id
and t.status = 'completed'
and date(t.end_time) >= p_day1
and date(t.end_time) < p_day2 + 1
and p.status = 'completed';
return res;
end;
$$
language plpgsql;
```

	QUERY PLAN	
	text	
1	Result (cost=0.00..0.26 rows=1 width=32) (actual time=0.349..0.350 rows=1 loops=...	
2	Planning Time: 0.021 ms	
3	Execution Time: 0.357 ms	

- idx_trip_driver_completed_end_time is a partial composite index that matches the query's most selective predicates (driver + completed + time range), reducing scanned rows dramatically.

- idx_payment_trip_completed lets PostgreSQL join via trip_id while keeping only completed payments, reducing join work.

	QUERY PLAN	
	text	
1	Result (cost=0.00..0.26 rows=1 width=32) (actual time=0.349..0.350 rows=1 loops=...	
2	Planning Time: 0.021 ms	
3	Execution Time: 0.357 ms	

- `idx_trip_driver_completed_end_time` is a partial composite index that matches the query's most selective predicates (driver + completed + time range), reducing scanned rows dramatically.
- `idx_payment_trip_completed` lets PostgreSQL join via `trip_id` while keeping only completed payments, reducing join work.